1. What is Exception in Java?

- ☐ Exception is an **abnormal condition.**
- ☐ An exception is an **unexpected event (runtime error)** that occurs during program execution and interrupts the normal flow of the program.
- ☐ **Examples**: divide by zero, invalid array index, null value, etc.
- ☐ In Java, an exception is an **event that disrupts the normal flow of the program.**
- ☐ An exception is a problem that happens **at runtime (execution time).**
- ☐ An exception (or exceptional event) is a problem that arises during the execution of a program.
- ☐ When an Exception **occurs the normal flow of the program is disrupted** and the program/Application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.
- ☐ It is an object which is thrown at runtime.
- ☐ An exception can occur for many different reasons, below given are some scenarios where exception occurs.
    - A user has entered invalid data.
    - A file that needs to be opened cannot be found.
    - A network connection has been lost in the middle of communications or the JVM has run out of memory.
- ☐ Some of these exceptions are caused by **user** error, others by **programmer** error, and others by **physical resources** that have failed in some manner.

2. What is Exception Handling?

- ☐ The **Exception Handling in Java** is one of the powerful mechanism **to handle the runtime errors so that the normal flow of the application can be maintained.**
- ☐ Exception Handling is a mechanism to **handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.**

Advantage of Exception Handling

1. Maintains Normal Flow of Program

- ☐ Even if an error occurs, the program does not stop suddenly.
- ☐ It continues executing the remaining code.
  - Without exception handling → **program terminates**
  - With exception handling → **program continues smoothly**

2. Improves Readability & Maintenance

Error-handling code is separated using `try`, `catch`, `finally`, making the program easier to understand and manage.

3. Handles Runtime Errors

- ☐ It helps in **handling runtime errors** like:
  - Division by zero
  - File not found
  - Array index out of bounds

4. Prevents Program Crashing

- ☐ Instead of crashing, Java catches the exception and allows you to show a meaningful message.

5. Allows Custom Error Messages

- ☐ You can display user-friendly messages instead of system-generated errors.

Example:

```
try {
  int a = 10 / 0;
} catch (ArithmeticException e) {
  System.out.println("Cannot divide by zero!");
}
```

6. Resource Management (finally block)

- ☐ The `finally` block ensures important code (like closing files) always executes.

7. Supports Multiple Exception Handling

- ☐ You can handle different types of errors using multiple `catch` blocks.

8. Helps in Debugging

You can trace errors easily using methods like:

```
e.printStackTrace();
```

3. List out various built-in exceptions in JAVA and explain any one Exception class with suitable example.

Common Built-in Exceptions in Java

Built-in exceptions are broadly categorized into two types:

1. **Unchecked Exceptions** (**Runtime Exceptions**): These occur during the **execution of the program**, often due to **logical errors.** The compiler does not force you to handle them.
2. **Checked Exceptions:** These are **checked by the compiler at compile-time**. The programmer must either handle them using a **try-catch block** or declare them in the method signature using the **throws** keyword.

Exception Class	Type	Description
ArithmeticException	Unchecked	Thrown when an illegal arithmetic operation happens, such as division by zero.
NullPointerException	Unchecked	Occurs when attempting to use an object reference that has a null value.
ArrayIndexOutOfBoundsException	Unchecked	Thrown when an array is accessed with an illegal index (negative or beyond size).

NumberFormatException	Unchecked	Raised when a method fails to convert a string into a numeric format.
FileNotFoundException	Checked	Occurs when an attempt to access a file fails because the file does not exist.
IOException	Checked	Thrown when an input-output operation is failed or interrupted.
ClassNotFoundException	Checked	Raised when an application tries to load a class through its string name but cannot find it.
InterruptedException	Checked	Thrown when a thread that is sleeping or waiting is interrupted by another thread.

Detailed Explanation: ArithmeticException

- The **ArithmeticException** is a built-in **unchecked exception** in Java that is thrown when an exceptional condition has occurred in an **arithmetic operation**.
- The most common trigger for this exception is division by zero with integers.

The following Java example demonstrates how to catch and handle an ArithmeticException using a try-catch block:

```
public class ArithmeticExceptionExample {
  public static void main(String[] args) {
    try {
      int a = 30;
      int b = 0;
      // This line will throw ArithmeticException as we cannot divide by zero
      int result = a / b;
      System.out.println("Result: " + result);
    } catch (ArithmeticException e) {
```

```
        // This block will execute when the exception is thrown
        System.out.println("Error: Cannot divide a number by zero.");
    }
    System.out.println("Rest of the code continues...");
  }
}
```

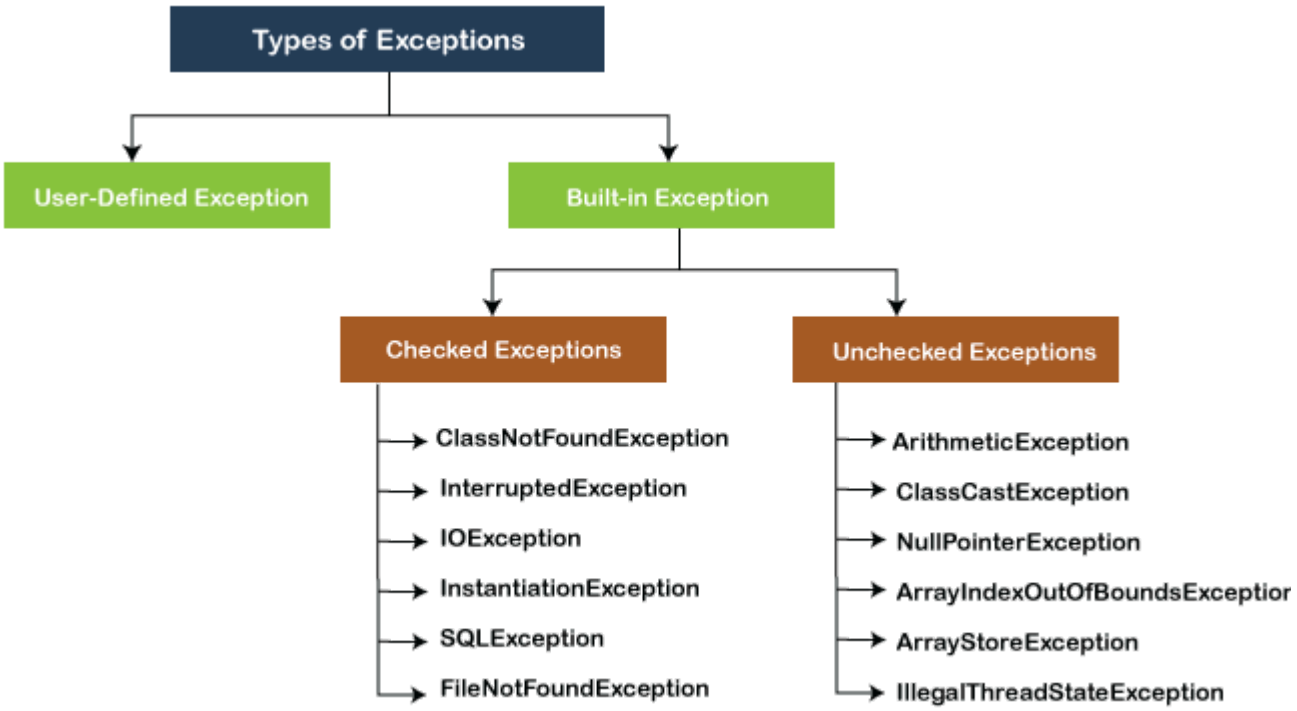
Output:

```
Error: Cannot divide a number by zero.
Rest of the code continues...
```

4. Types of Exception in Java

Exceptions can be categorized into two ways:

1. **Built-in Exceptions**
    - Checked Exception (Compile-time Exceptions)
    - Unchecked Exception
2. **User-Defined Exceptions**
3. **Eror**



1. Built-in Exception

- Exceptions that are already available in **Java libraries** are referred to as **built-in exception**.
- These exceptions are able to define the error situation so that we can understand the reason for getting this error.
- It can be categorized into two broad categories, i.e., **checked exceptions and unchecked exception**.

Checked Exception

- **Checked** exceptions are called **compile-time exceptions** because these exceptions are **checked at compile-time by the compiler.**
- The compiler ensures whether the programmer handles the exception or not.
- The programmer must either handle it using a **try-catch block or declare it using the throws keyword in the method signature.**
- These typically represent conditions outside the program's immediate control, such as network or file system issues.
- For example, **IOException, SQLException**, etc. Checked exceptions are checked at compile-time.
    - **IOException:** Occurs during failed or interrupted input/output operations, such as

reading a file.

- **SQLException**: Thrown when there is an error in database access or other database issues.
- **ClassNotFoundException**: Raised when an application tries to load a class through its string name but cannot find the definition.
- **InterruptedException**: Thrown when a thread is interrupted while it is waiting or sleeping.

Unchecked Exceptions

- ☐ **Unchecked exceptions** occur during program execution (**runtime**) and are **not checked by the compiler.**
- ☐ They usually result from programming logic errors or improper use of an API.
- ☐ While not mandatory to handle, catching them can prevent the application from crashing. All unchecked exceptions are subclasses of the RuntimeException class.

Examples:

1. ArithmeticException
2. NullPointerException
3. ArrayIndexOutOfBoundsException
4. NumberFormatException

- ☐ **ArithmeticException**: Thrown for exceptional arithmetic conditions, most commonly division by zero.
- ☐ **NullPointerException**: Occurs when attempting to use an object reference that has a null value.
- ☐ **ArrayIndexOutOfBoundsException**: Thrown to indicate that an array has been accessed with an illegal index (e.g., negative or exceeding array size).
- ☐ **NumberFormatException**: Raised when a method fails to convert a string into a numeric format

2. User-defined Exception

- In Java, we can write our own exception class by extends the Exception class.
- We can throw our own exception on a particular condition using the throw keyword.
- For creating a user-defined exception, we should have basic knowledge of **the try-catch** block and **throw** keyword.

3. Errors

- ☐ In Java, an **Error** represents **serious, irrecoverable problems** that a reasonable application should not try to catch.
- ☐ They are typically related to **environment or JVM failures** rather than the program logic itself.
  - **OutOfMemoryError**: Thrown when the JVM cannot allocate more memory for an object.
  - **StackOverflowError**: Occurs when an application recurses too deeply, exhausting the stack space.
  - **VirtualMachineError**: The superclass for errors indicating the JVM has broken or run out of resources

5. Write a program to rise and handle divide by zero exception.

```
class DivideByZeroExample {
  public static void main(String[] args) {
    try {
      int a = 10;
      int b = 0;

      int result = a / b;   // This will raise exception

      System.out.println("Result = " + result);
    } catch (ArithmeticException e) {
      System.out.println("Exception caught: Cannot divide by zero");
    }

    System.out.println("Program continues after handling exception");
  }
```

}

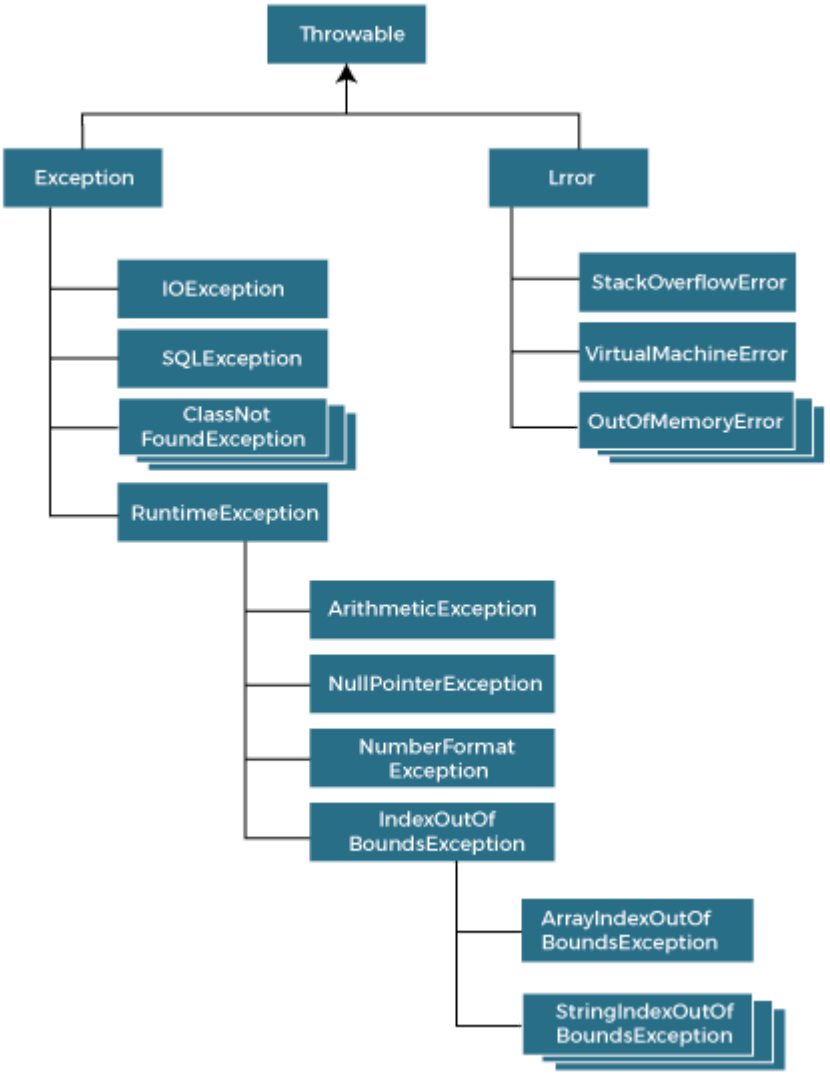
Output:

Exception caught: Cannot divide by zero

Program continues after handling exception

6. Explain the exception hierarchy.

- ☐ The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



1. Throwable (Super Class)

- ☐ Root class of all exceptions and errors
- ☐ Only objects of this class (or its subclasses) can be thrown

2. Exception Class

- ☐ Represents conditions that a program can handle
- ☐ Divided into:
  - ☐ Checked Exceptions (compile-time)
  - ☐ Unchecked Exceptions (runtime)

3. Checked Exceptions

- ☐ Must be handled using `try-catch` or `throws`
- ☐ Examples:
  - ☐ IOException
  - ☐ SQLException
  - ☐ ClassNotFoundException

4. RuntimeException (Unchecked Exceptions)

- ☐ Occur at runtime
- ☐ Not mandatory to handle

Examples:

- ☐ ArithmeticException
- ☐ NullPointerException
- ☐ ArrayIndexOutOfBoundsException

5. Error Class

- ☐ Represents serious system-level problems
- ☐ Cannot be handled easily

Examples:

- ☐ OutOfMemoryError
- ☐ StackOverflowError

(Note: in detail i already describe every topic in types of exceptions. Please refer to that according to the question weightage).

7. Difference Between Checked and Unchecked Exception

S.No	Checked Exception	Unchecked Exception
1.	These exceptions are **checked at compile time.** These exceptions are handled at compile time too.	These exceptions are just opposite to the checked exceptions. These exceptions are **not checked and handled at compile time.**
2.	These exceptions are direct subclasses of exception but not extended from RuntimeException class.	They are the direct subclasses of the RuntimeException class.
3.	The code **gives a compilation error** in the case when a method throws a checked exception. The **compiler is not able to handle the exception** on its own.	The code **compiles without any error** because the exceptions escape the notice of the compiler. **These exceptions are the results of user-created errors** in programming logic.
4.	These exceptions mostly occur when the probability of failure is too high.	These exceptions occur mostly due to **programming mistakes.**
5.	Common checked exceptions include **IOException, DataAccessException, InterruptedException, etc.**	Common unchecked exceptions include **ArithmeticException, InvalidClassException, NullPointerException, etc.**
6.	These exceptions are propagated using the **throws** keyword.	These are **automatically propagated.**

7.	It is required to provide the **try-catch and try-finally block to handle the checked exception**.	In the case of unchecked exception it is **not mandatory.**

8. Write exception handling mechanisms in JAVA.

Note: While answering this question, make sure to cover **try, catch, multiple catch, throw, throws, finally, and custom exception(all of this are exception handling mechanisms)**, referring to the answers I have already prepared in my notes.

9. Explain try, catch and finally with example.

try block

- ☐ try block Contains code that **may cause an exception**
- ☐ it Must be followed by at least one `catch` or `finally`
- ☐ The try statement allows you to define a block of code to be tested for errors while it is being executed.
- ☐ Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.
- ☐ If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Syntax of Java try-catch

```
Try
{
    //code that may throw an exception
}
```

```
catch(Exception_class_Name ref){}
```

catch block

- The catch statement Used to **handle the exception**
- You can write multiple `catch` blocks for different exceptions
- The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.
- Java catch block is used to handle the Exception by declaring the type of exception within the parameter.
- The declared exception must be the parent class exception (i.e., Exception) or the generated exception type.
- The catch block must be used after the try block only.
- You can use multiple catch block with a single try block.

finally Block

- This block contains code that always executes, regardless of whether an exception was thrown or caught.
- It is primarily used for cleanup activities, such as closing file streams or database connections

Syntax:

```
finally {
// code that always executes
}
```

Example:

```
class Example {
  public static void main(String[] args) {
    try {
      int a = 10;
```

```
            int b = 0;
            int c = a / b;   // Exception occurs here
            System.out.println("Result = " + c);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        } finally {
            System.out.println("Finally block always executes");
        }

        System.out.println("Program continues...");
    }
}
```

Output:

```
Error: Cannot divide by zero
Finally block always executes
Program continues…
```

10. Demonstrate how you can handle different types of exception separately.

- To handle different types of exceptions separately in Java, you use **multiple catch blocks,** where **each block handles a specific exception type.**

When an exception occurs in the try block:

- Java checks each catch block from top to bottom
- The first matching catch is executed
- Remaining catch blocks are skipped

Example:

```
public class MultipleExceptionDemo {
  public static void main(String[] args) {
```

```
        try {
            int a = 10;
            int b = 0;
            int result = a / b;   // ArithmeticException

            int arr[] = new int[5];
            arr[10] = 50;       // ArrayIndexOutOfBoundsException
            String str = null;
            System.out.println(str.length());  // NullPointerException
        }
        catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Array index is out of range");
        }
        catch (NullPointerException e) {
            System.out.println("Error: Null value found");
        }
        catch (Exception e) {
            System.out.println("General Exception occurred");
        }
        System.out.println("Program continues...");
    }
}
```

Output:

Error: Cannot divide by zero

Program continues...

11. Demonstrate use of try-catch construct in case of hierarchical Exception Handling. (i.e handling various exception belongs to the exception hierarchy)

- In hierarchical exception handling, **multiple catch blocks are used to handle different types of exceptions thrown from a single try block.**
- The most critical rule is that specific (**subclass**) exceptions must be caught before general (**superclass**) exceptions.
- In this case, we handle **different exceptions from the same hierarchy using multiple catch blocks**, arranged from **specific to general**.

Example: Hierarchical Exception Handling

```
public class HierarchicalExceptionDemo {
  public static void main(String[] args) {

    try {
      int a = 10 / 0;  // ArithmeticException

      String str = null;
      System.out.println(str.length()); // NullPointerException

    }
    catch (ArithmeticException e) {
      System.out.println("Arithmetic Exception handled");
    }
    catch (NullPointerException e) {
      System.out.println("Null Pointer Exception handled");
    }
    catch (RuntimeException e) {
```

```
            System.out.println("Runtime Exception handled");
        }
        catch (Exception e) {
            System.out.println("General Exception handled");
        }

        System.out.println("Program continues...");
    }
}
```

Output:

Arithmetic Exception handled

Program continues…

Explanation

- ArithmeticException and NullPointerException are **child classes** of RuntimeException
- RuntimeException is a child of Exception
- Java checks catch blocks **from top to bottom**
- The **most specific exception must come first**, and the most general (Exception) last

12. Multiple Catch Blocks

- A try block can be followed by multiple catch blocks.
- A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.
- At a time only one exception occurs and at a time only one catch block is executed.

The syntax for multiple catch blocks looks like the following –

Syntax

```
try {
  // Protected code
} catch (ExceptionType1 e1) {
  // Catch block
} catch (ExceptionType2 e2) {
  // Catch block
}
```

Example

```
public class MultipleCatchBlock1 {
  public static void main(String[] args) {
      try{
         int a[]=new int[5];
         a[5]=30/0;
        }
        catch(ArithmeticException e)
          {
           System.out.println("Arithmetic Exception occurs");
          }
        catch(ArrayIndexOutOfBoundsException e)
          {
           System.out.println("ArrayIndexOutOfBounds Exception occurs");
          }
        catch(Exception e)
          {
           System.out.println("Parent Exception occurs");
          }
        System.out.println("rest of the code");
  }
}
```

13. Java Nested try block

- In Java, using a try block inside another try block is permitted. It is called as nested try block.
- For example, the **inner try block** can be used to handle **ArrayIndexOutOfBoundsException** while the **outer try block** can handle the **ArithemeticException** (division by zero).
- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.

Syntax:

```
....
//main try block
try
{
    //try catch block within another try block
  try
  {
//try catch block within nested try block
  }
  catch(Exception e1)
  {
//exception message
  }
}   catch(Exception e3)
{
//exception message
}
```

Java Nested try Example

```
public class NestedTryBlock{
 public static void main(String args[]){
```

```
//outer try block
 try{
 //inner try block 1
  try{
   System.out.println("going to divide by 0");
   int b =39/0;
  }
  //catch block of inner try block 1
  catch(ArithmeticException e)
  {
   System.out.println(e);
  }
  System.out.println("other statement");
 }
 //catch block of outer try block
 catch(Exception e)
 {
  System.out.println("handled the exception (outer catch)");
 }

 System.out.println("normal flow..");
}
}
```

14. Explain how to throw, catch and handle Exceptions

- ☐ An exception is an **error** that occurs **during the execution of a program**.
- ☐ Java provides a mechanism to **throw**, **catch**, and **handle** exceptions so that the program can **continue its normal flow.**

1. Throwing an Exception (throw)

- The throw keyword is used to **explicitly create and throw an exception**
- It is used when **we want to generate an exception manually**

Example:

```
public class Test {
  public static void main(String[] args) {
    int age = 15;
    if (age < 18) {
      throw new ArithmeticException("Not eligible to vote");
    }

    System.out.println("Eligible to vote");
  }
}
```

2. Catching an Exception (try-catch)

- The try block contains code that may cause an exception
- The catch block handles the exception

Example:

```
public class Test {
  public static void main(String[] args) {
    try {
      int a = 10 / 0;
    } catch (ArithmeticException e) {
      System.out.println("Cannot divide by zero");
    }
  }
}
```

3. Handling Exceptions

- Handling means managing the exception so the program does not crash
- This is done using try-catch, finally, and proper messages

Example:

```
public class Test {
  public static void main(String[] args) {
    try {
      int a = 10 / 0;
    } catch (ArithmeticException e) {
      System.out.println("Error handled: " + e.getMessage());
    } finally {
      System.out.println("Program continues safely");
    }
  }
}
```

Java throws keyword

- A throws statement lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause. If they are not, a compile-time error will result.

Syntax of Java throws

```
return_type method_name() throws exception_class_name{
//method code
}
```

Example

```
import java.io.IOException;
class Testthrows1{
 void m() throws IOException {
  throw new IOException("device error");//checked exception
 }
 void p(){
  try{
   m();
  }catch(Exception e){System.out.println("exception handled");}
 }
 public static void main(String args[]){
  Testthrows1 obj=new Testthrows1();
  obj.p();
 }
}
```

15. What is the keyword "throw" used for? What is the keyword "throws" used for?

1. throw Keyword

Definition:

- ☐ The throw keyword is used to **explicitly throw an exception** from a method or block of code.
- ☐ To manually generate an exception
- ☐ To transfer control to the nearest catch block
- ☐ Used when the programmer wants to handle abnormal conditions

Key Points

- ☐ Used **inside a method or block**

- ☐ Followed by an **exception object**
- ☐ Only **one exception** can be thrown at a time
- ☐ After throw, execution stops and control shifts to catch

Syntax :

```
throw new exception_class("error message");
```

Example :

```
throw new IOException("sorry device error");
```

throw keyword Example

```
public class TestThrow1 {
//function to check if person is eligible to vote or not
  public static void validate(int age) {
        if(age<18) {
        //throw Arithmetic exception if not eligible to vote
              throw new ArithmeticException("Person is not eligible to vote");
        }
        else {
          System.out.println("Person is eligible to vote!!");
        }
    }
    //main method
    public static void main(String args[]){
      //calling the function
        validate(13);
      }
  }
```

2. throws Keyword

Definition

- ☐ The `throws` keyword is used to **declare exceptions** that a method may throw during execution.

Purpose

- ☐ To inform the caller about possible exceptions
- ☐ To delegate exception handling responsibility to the calling method
- ☐ Mainly used with **checked exceptions** (like IOException)

Key Points

- ☐ Used in **method declaration (signature)**
- ☐ Followed by **exception class names**
- ☐ Can declare **multiple exceptions** separated by commas
- ☐ Does **not throw exception**, only declares

Syntax

```
returnType methodName() throws ExceptionType1, ExceptionType2 {
  // code
}
```

Example

```
import java.io.*;
class Test {
  static void readFile() throws IOException {
     FileReader file = new FileReader("abc.txt");
  }
  public static void main(String[] args) {
     try {
```

```
            readFile();
        } catch (IOException e) {
            System.out.println("File not found");
        }
    }
}
```

Here, `throws` only **declares**, actual handling is done in `main()`

16. Difference between throw and throws

Sr. no.	Basis of Differences	throw	throws
1.	Definition	Java throw keyword is used throw an exception explicitly in the code, inside the function or the block of code.	Java throws keyword is used in the method signature to declare an exception which might be thrown by the function while the execution of the code.
2.	Type of exception Using throw keyword, we can only propagate unchecked exception i.e., the checked exception cannot be propagated using throw only.	Using throws keyword, we can declare both checked and unchecked exceptions. However, the throws keyword can be used to propagate checked exceptions only.	

3.	Syntax	The throw keyword is followed by an instance of Exception to be thrown.	The throws keyword is followed by class names of Exceptions to be thrown.
4.	Declaration	throw is used within the method.	throws is used with the method signature.
5.	Internal implementation	We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.

17. Java finally block

- **Java finally block** is a block used to execute important code such as closing the connection, etc.
- Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

The finally block follows the try-catch block.

- finally block in Java can be used to put "**cleanup**" code such as closing a file, closing connection, etc.
- The important statements to be printed can be placed in the finally block.

```
public class ExcepTest {


  public static void main(String args[]) {
    int a[] = new int[2];
```

```
  try {
    System.out.println("Access element three :" + a[3]);
  } catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Exception thrown  :" + e);
  }finally {
    a[0] = 6;
    System.out.println("First element value: " + a[0]);
    System.out.println("The finally statement is executed");
  }
}}
```

This will produce the following result –

Output

```
Exception thrown  :java.lang.ArrayIndexOutOfBoundsException: 3
First element value: 6
The finally statement is executed
```

18. Difference between final, finally and finalize.

The final, finally, and finalize are keywords in Java that are used in exception handling.

final - access modifier

finally - block in Exception Handling

finalize - method of object class.

Sr. no.	final	finally	finalize
1.	final is the keyword and access modifier which is used to apply restrictions on a class, method or variable.	finally is the block in Java Exception Handling to execute the important code whether the exception occurs or not.	finalize is the method in Java which is used to perform clean up processing just before object is garbage collected.

2.	Final keyword is used with the classes, methods and variables.	Finally block is always related to the try and catch block in exception handling.	finalize() method is used with the objects.
3.	(1) Once declared, final variable becomes constant and cannot be modified. (2) final method cannot be overridden by sub class. (3) final class cannot be inherited.	(1) finally block runs the important code even if exception occurs or not. (2) finally block cleans up all the resources used in try block	finalize method performs the cleaning activities with respect to the object before its destruction.
4.	Final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed. It's execution is not dependant on the exception.	finalize method is executed just before the object is destroyed.

19. Custom Exception

- In Java, we can create our own exceptions that are derived classes of the Exception class.
- Creating our own Exception is known as custom exception or user-defined exception.
- Basically, Java custom exceptions are used to customize the exception according to user need.
- In order to create custom exception, we need to extend Exception class that belongs to java.lang package.

```
// class representing custom exception
class InvalidAgeException  extends Exception
{
  public InvalidAgeException (String str)
  {
    // calling the constructor of parent Exception
```

```
        super(str);
    }
}

// class that uses custom exception InvalidAgeException
public class TestCustomException1
{

    // method to check the age
    static void validate (int age) throws InvalidAgeException{
      if(age < 18){

        // throw an object of user defined exception
        throw new InvalidAgeException("age is not valid to vote");
    }
      else {
        System.out.println("welcome to vote");
        }
     }

    // main method
    public static void main(String args[])
    {
        try
        {
            // calling the method
            validate(13);
        }
        catch (InvalidAgeException ex)
```

```
        {
            System.out.println("Caught the exception");

            // printing the message from InvalidAgeException object
            System.out.println("Exception occured: " + ex);
        }

        System.out.println("rest of the code...");
    }
}
```